The Emacs Series
Exploring dash.el

This is the second part of the series of articles on dash.el. In this article, we will continue to explore the functions provided by dash.el — a list library for Emacs released under the GNU General Public License v3.0.

The dash.el library has been written by Magnar Sveen, and the latest release is v2.16.0. The source code is available at *https://github.com/magnars/dash.el*.

Installation

The dash.el package is available in Milkypostman's Emacs Lisp Package Archive (MELPA) and in the Marmalede repo. You can install the package using the following command in GNU Emacs:

```
M-x package-install dash
```

If you want the function combinators, then you should also run the following:

```
M-x package-install dash-functional
```

The other method of installation is to copy the dash.el source file to your Emacs load path and use it.

In order to get syntax highlighting of dash functions in Emacs buffers, you can add the following code to your Emacs initialisation settings:

```
(eval-after-load 'dash '(dash-enable-font-lock))
```

The API functions available in dash.el are prefixed with a dash ("-"). In this article, we will review the unfolding, predicate, partitioning, indexing and set operations in dash.el.

Unfolding

The unfolding operations produce a list from an initial or seed value. This operation is the opposite of reduction functions. The '-iterate' function takes three arguments, - *a* function, an initial value and a count. It applies the function to the initial value to produce a new value, and iterates with this new value to return a new list. A couple of examples are shown below:

```
(-iterate function initial count) ;; Syntax

(-iterate '1+ 1 5)
(1 2 3 4 5)

(-iterate (lambda (x) (+ x x)) 1 5)
(1 2 4 8 16)
```

The '-unfold' function creates a new list by applying a function to the seed value. The function should have a check for termination; otherwise, it will build an infinite list.

```
(-unfold function seed);; Syntax

(-unfold (lambda (x) (unless (= x 0) (cons x (1- x)))) 5)
(5 4 3 2 1)


(--unfold (when it (cons it (cdr it))) '(1 2 3 4))
((1 2 3 4) (2 3 4) (3 4) (4))
```